

# SplitSync

## Architecture & Implementation Blueprint

Target: Desktop Web (1440px) • Tech Stack: Next.js, Supabase, Vercel AI SDK

This document outlines the technical architecture required to execute the high-density SplitSync dashboard. The focus is on strictly typing the data models, optimizing the settlement flows, and orchestrating the AI pipeline for seamless receipt parsing.

### 1. System Architecture & Data Schema

At the core of the Next.js application, Supabase will handle relational data and authentication. Establishing a strict PostgreSQL schema is critical for the multi-currency functionality and the audit trail.

| Table          | Core Columns                                                                               | Purpose & Relationships                                                                         |
|----------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| users          | id, email, full_name, avatar_url, default_currency                                         | Stores user profiles. Links to auth.users.                                                      |
| groups         | id, name, base_currency, created_by, created_at                                            | The central node. All expenses resolve to the group's <i>base_currency</i> .                    |
| group_members  | group_id, user_id, joined_at                                                               | Join table for group access and participant listing.                                            |
| expenses       | id, group_id, paid_by (user_id), amount, currency, base_amount, exchange_rate, receipt_url | Captures the original transaction context and the normalized numerical value for the algorithm. |
| expense_splits | expense_id, user_id, split_type (equal, exact, percentage), amount_owed                    | Defines how an individual expense is divided among the group members.                           |
| settlements    | id, group_id, from_user, to_user, amount, status (pending, paid)                           | The optimized output generated by the graph algorithm.                                          |

## 2. Debt Optimization (Graph Algorithm)

---

Minimizing cash flow across a group is fundamentally a directed flow network optimization. To power the "Minimized Debt Matrix" in the right-hand sidebar, the backend must process node balances to reduce the total number of transaction edges.

### Algorithmic Approach

A greedy approach processes the nodes effectively. By establishing an array of net balances (where positive balances denote creditors and negative balances denote debtors), sorting them, and continuously satisfying the largest absolute values, the system resolves the network efficiently. This logic performs optimally in a standard server environment without heavy memory overhead.

**TypeScript Edge Function Implementation Strategy:**

```

// Compute net balances map: { userId: netAmount }
const balances = new Map<string, number>();

// Separate and sort
const creditors = Array.from(balances.entries())
  .filter(([_, amt]) => amt > 0)
  .sort((a, b) => b[1] - a[1]);

const debtors = Array.from(balances.entries())
  .filter(([_, amt]) => amt < 0)
  .sort((a, b) => a[1] - b[1]); // Sort ascending (most negative first)

const settlements = [];
let i = 0, j = 0;

// Greedy resolution
while (i < debtors.length && j < creditors.length) {
  const [debtorId, debtAmt] = debtors[i];
  const [creditorId, credAmt] = creditors[j];

  const minAmount = Math.min(Math.abs(debtAmt), credAmt);

  settlements.push({
    from: debtorId,
    to: creditorId,
    amount: Number(minAmount.toFixed(2))
  });

  debtors[i][1] += minAmount;
  creditors[j][1] -= minAmount;

  if (Math.abs(debtors[i][1]) < 0.01) i++;
  if (creditors[j][1] < 0.01) j++;
}

```

### 3. AI Receipt OCR Pipeline

The 900px modal requires rapid, structured extraction. Instead of a brittle regex-based OCR pipeline, passing the visual data through a multimodal LLM ensures robust parsing, even with crumpled or faded receipts.

#### Execution Flow:

1. **Client (Next.js):** The user drops the image. The client uploads it directly to a Supabase Storage bucket, generating a temporary signed URL.

2. **Server Action:** The client triggers a Server Action, passing the signed URL.
3. **Vercel AI SDK:** The action utilizes `generateObject` with a strictly defined Zod schema to enforce type safety on the LLM's output.

**Strict Schema Definition:** Leveraging TypeScript interfaces guarantees that the returned JSON maps perfectly to the interactive split table state, preventing frontend runtime errors.

```
import { generateObject } from 'ai';
import { z } from 'zod';
import { openai } from '@ai-sdk/openai'; // Or Claude/Anthropic provider

const receiptSchema = z.object({
  merchantName: z.string(),
  totalAmount: z.number(),
  date: z.string(),
  category: z.string(),
  lineItems: z.array(z.object({
    description: z.string(),
    price: z.number()
  }))
});

// Inside the Next.js API route or Server Action
const { object } = await generateObject({
  model: openai('gpt-4o'),
  schema: receiptSchema,
  messages: [
    {
      role: 'user',
      content: [
        { type: 'text', text: 'Extract the receipt details into the exact schema provided' },
        { type: 'image', image: new URL(signedReceiptUrl) }
      ]
    }
  ]
});
```

## 4. Multi-Currency Normalization

To maintain a clean audit log and accurate Recharts analytics, foreign transactions must be converted at the time of entry.

**Data Integrity Note:** Never overwrite the `original_amount`. If a user spends €50 on a trip where the base currency is USD, the database must store `amount : 50, currency : 'EUR'`, and `base_amount : 54.20`. The optimization algorithm and UI analytics must read exclusively from the `base_amount`.

## 5. Recommended Implementation Phases

|                                  |                                                                                                                                                                   |
|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Phase 1: Foundation</b>       | Initialize Next.js App Router, configure Tailwind with Slate/Emerald/Indigo palette, and deploy the Supabase schema. Build standard CRUD for groups and expenses. |
| <b>Phase 2: Logic &amp; Edge</b> | Implement the greedy optimization algorithm in a server utility. Connect external exchange rate APIs for multi-currency handling upon expense creation.           |
| <b>Phase 3: AI &amp; Polish</b>  | Integrate the Vercel AI SDK for the receipt upload modal. Build the Recharts analytics dashboard reading from the unified audit log.                              |